

ĐẠI HỌC BÁCH KHOA HÀ NỘI
Khoa Toán - Tin



BÁO CÁO BÀI TẬP LỚN
KỸ THUẬT LẬP TRÌNH
(MI3310)

Đề tài:

**Xây dựng chương trình
hỗ trợ thi trắc nghiệm đơn giản**

Giảng viên hướng dẫn: TS. Vũ Thành Nam
Mã lớp học: 169312
Nhóm sinh viên thực hiện:

- | | | | |
|---|------------------|-----------|-------------------------------|
| 1 | Nguyễn Mạnh Hùng | 202419063 | Hệ thống thông tin quản lý 02 |
| 2 | Nguyễn Đức Thành | 202419099 | Hệ thống thông tin quản lý 02 |
| 3 | Nguyễn Hồng Vân | 202419119 | Hệ thống thông tin quản lý 02 |

Hà Nội, Tháng 6 2026

Lời mở đầu

Trong bối cảnh giáo dục ngày càng chú trọng đến việc ứng dụng công nghệ thông tin vào công tác đánh giá và kiểm tra, các hệ thống thi trắc nghiệm tự động đã trở thành một công cụ không thể thiếu tại nhiều cơ sở đào tạo. Từ những nền tảng trực tuyến quy mô lớn cho đến các phần mềm nội bộ đơn giản, điểm chung của chúng đều nằm ở khả năng tự động hoá quy trình ra đề, thu bài và chấm điểm — giảm thiểu công sức thủ công và nâng cao tính khách quan.

Xuất phát từ yêu cầu của môn học **Kỹ thuật Lập trình (MI3310)**, nhóm chúng em lựa chọn đề tài “*Xây dựng chương trình hỗ trợ thi trắc nghiệm đơn giản*” với mục tiêu vận dụng tổng hợp các kiến thức đã học vào một sản phẩm phần mềm có tính thực tiễn. Đây không đơn thuần là bài tập lập trình thông thường — đề tài đặt ra yêu cầu thiết kế một hệ thống hoàn chỉnh, bao gồm quản lý ngân hàng câu hỏi, sinh đề ngẫu nhiên, tiến hành thi có đếm giờ, chấm điểm tự động và lưu trữ lịch sử kết quả.

Điểm thách thức đặc biệt của đề tài nằm ở ràng buộc kỹ thuật: toàn bộ chương trình được xây dựng bằng ngôn ngữ **C++** thuần, không sử dụng các container có sẵn của thư viện chuẩn như `std::vector` hay `std::list`. Thay vào đó, nhóm tự cài đặt cấu trúc danh sách liên kết đơn bằng con trỏ, tự quản lý bộ nhớ động và tự triển khai tất cả các thuật toán cốt lõi. Điều này buộc mỗi thành viên phải nắm vững các nguyên lý nền tảng của lập trình hệ thống thay vì phụ thuộc vào các giải pháp trừu tượng hoá.

Báo cáo này trình bày toàn bộ quá trình phân tích, thiết kế, cài đặt và kiểm thử hệ thống. Nội dung được tổ chức theo cấu trúc khoa học: từ phân tích yêu cầu, thiết kế kiến trúc module, đến đánh giá kết quả kiểm thử và tổng kết các kỹ thuật đã vận dụng. Nhóm hy vọng báo cáo không chỉ phản ánh trung thực kết quả công việc mà còn là tài liệu tham khảo hữu ích cho các bạn sinh viên quan tâm đến lập trình hệ thống với C++.

Nhóm xin gửi lời cảm ơn chân thành đến **TS. Vũ Thành Nam** — giảng viên hướng dẫn môn Kỹ thuật Lập trình — vì những bài giảng sâu sắc và những góc nhìn thực tiễn đã định hướng cho chúng em trong suốt quá trình thực hiện đề tài. Dù đã cố gắng hết sức, báo cáo chắc chắn còn nhiều điểm cần cải thiện; nhóm rất mong nhận được sự góp ý từ thầy và hội đồng để hoàn thiện hơn trong tương lai.

Hà Nội, tháng 6 năm 2026

Nhóm sinh viên thực hiện

Nguyễn Mạnh Hùng — Nguyễn Đức Thành — Nguyễn Hồng Vân

Danh sách hình vẽ

Danh sách bảng

1.1	Công nghệ và công cụ sử dụng trong dự án	9
1.2	Danh sách thành viên nhóm	9
1.3	Phân công nhiệm vụ giữa các thành viên	10
2.1	Các thao tác trên danh sách liên kết đơn QuestionBank	14
2.2	Cấu trúc header/source của các module trong dự án	17
2.3	Tổng kết liên hệ lý thuyết và cài đặt trong dự án	18

Mục lục

Lời mở đầu	1
Danh sách hình vẽ	2
Danh sách bảng	3
1 Giới thiệu	7
1.1 Bối cảnh	7
1.2 Lý do lựa chọn bài toán	7
1.3 Mô tả bài toán	8
1.4 Mục tiêu của chương trình	8
1.5 Phạm vi thực hiện	8
1.6 Công nghệ và ngôn ngữ lập trình sử dụng	9
1.7 Phân công công việc	9
1.7.1 Thành viên thực hiện	9
1.7.2 Phân công chi tiết nhiệm vụ	10
2 Cơ sở lý thuyết	11
2.1 Nguyên lý thiết kế chương trình theo module	11
2.1.1 Tư tưởng module hoá (Modularity)	11
2.1.2 Thiết kế từ trên xuống (Top-down Design)	11
2.1.3 Phong cách lập trình tốt	12
2.2 Con trỏ và quản lý bộ nhớ động trong C++	12
2.2.1 Bộ nhớ tĩnh và bộ nhớ động	12
2.2.2 Rò rỉ bộ nhớ (Memory Leak)	13
2.2.3 Constructor và Destructor	13
2.3 Danh sách liên kết đơn (Singly Linked List)	13
2.3.1 Khái niệm và cấu trúc	13
2.3.2 Ưu điểm so với mảng tĩnh	14
2.3.3 Các thao tác cơ bản	14
2.3.4 Cài đặt thêm và xoá node	14
2.4 Thuật toán xáo trộn Fisher-Yates	14
2.4.1 Nguyên lý hoạt động	14
2.4.2 Độ phức tạp	15
2.4.3 Ứng dụng trong dự án	15
2.5 Xử lý File I/O trong C++	15
2.5.1 Các lớp stream cơ bản	15
2.5.2 Định dạng dữ liệu	15
2.5.3 Đọc file với xử lý lỗi	16
2.6 Lập trình hướng đối tượng trong C++	16

2.6.1	Class, đóng gói và trừu tượng hoá	16
2.6.2	Tách khai báo và cài đặt (Header/Source)	16
2.6.3	Validate input và lập trình phòng ngừa	17
2.7	Tổng kết	17
3	Phân tích Yêu cầu hệ thống	19
3.1	Khảo sát bài toán	19
3.2	Yêu cầu chức năng	19
3.2.1	Quản lý ngân hàng câu hỏi	19
3.2.2	Cấu hình và sinh đề thi	19
3.2.3	Tiến hành thi và đếm giờ thời gian thực	19
3.2.4	Chấm điểm tự động và hiển thị kết quả	19
3.2.5	Xem lịch sử thi	19
3.3	Yêu cầu phi chức năng	19
3.4	Mô tả luồng hoạt động của chương trình	19
4	Thiết kế hệ thống	20
4.1	Kiến trúc tổng thể chương trình	21
4.1.1	Sơ đồ phân chia module	21
4.1.2	Luồng dữ liệu giữa các module	21
4.1.3	Sơ đồ luồng điều hướng Menu	21
4.2	Thiết kế cấu trúc dữ liệu	21
4.2.1	Cấu trúc file questions.txt	21
4.2.2	Cấu trúc file history.txt	21
4.2.3	Quy tắc định dạng và tách trường dữ liệu	21
4.3	Thiết kế các module chức năng	21
4.3.1	Module Menu	21
4.3.2	Module Ngân hàng câu hỏi	21
4.3.3	Module Thi	21
4.3.4	Module Lịch sử thi	21
4.3.5	Hàm main	21
5	Cài đặt và xây dựng chương trình	22
5.1	Cấu trúc thư mục chương trình	22
6	Kiểm thử và Đánh giá kết quả	23
6.1	Chiến lược kiểm thử	23
6.2	Tình huống kiểm thử	23
6.2.1	Test Case-01: Đăng nhập đúng – Admin	23
6.2.2	Test Case-02: Đăng nhập đúng – Sinh viên	23
6.2.3	Test Case-03: Đăng nhập sai mật khẩu	23
6.2.4	Test Case-04: Thêm câu hỏi mới	23
6.2.5	Test Case-05: Sinh đề và xáo trộn câu hỏi/đáp án	23
6.2.6	Test Case-06: Làm bài và chấm điểm chính xác	23
6.2.7	Test Case-07: Tự động thu bài khi hết giờ	23
6.2.8	Test Case-08: Xem lịch sử thi nhiều lần	23
6.2.9	Test Case-09: Nhập sai định dạng (bẫy lỗi)	23
6.2.10	Test Case-10: Kiểm thử rò rỉ bộ nhớ (Valgrind)	23
6.3	Hình ảnh chụp kết quả thực thi	23
6.4	Đánh giá kết quả kiểm thử	23

7	Kết luận và hướng phát triển	24
7.1	Kết quả đạt được	24
7.2	Tổng kết các kỹ thuật đã được vận dụng	24
7.2.1	Cấu trúc dữ liệu: Danh sách liên kết đơn	24
7.2.2	Quản lý bộ nhớ động: con trỏ, new/delete, tránh Memory Leak	24
7.2.3	Xử lý File I/O: đọc/ghi file text với ký tự phân tách ' '	24
7.2.4	Thuật toán xáo trộn Fisher-Yates	24
7.2.5	Lập trình hướng module: tách Header – Source – Data	24
7.2.6	Lập trình hướng đối tượng (OOP): Class, Constructor, Destructor	24
7.2.7	Lập trình phòng ngừa	24
7.2.8	Validate input và xử lý ngoại lệ trên Terminal	24
7.2.9	Quy chuẩn lập trình: Naming Convention, No Global Variables	24
7.3	Đánh giá ưu điểm và hạn chế của chương trình	24
7.4	Hướng phát triển và Nâng cấp tương lai	24
A	Mã nguồn hàm main()	26
B	Mô tả các hàm xử lý chính	27
C	Hướng dẫn cài đặt và sử dụng chương trình	28

Chương 1

Giới thiệu

1.1 Bối cảnh

Trong những năm gần đây, việc ứng dụng công nghệ thông tin vào giáo dục đã tạo ra những bước chuyển biến sâu sắc trong cách thức tổ chức kiểm tra và đánh giá năng lực người học. Các hệ thống thi trắc nghiệm điện tử không chỉ giúp tiết kiệm chi phí in ấn và thời gian chấm bài thủ công, mà còn đảm bảo tính công bằng, khách quan và có thể mở rộng quy mô dễ dàng [?]. Tại các trường đại học kỹ thuật, nơi khối lượng sinh viên lớn và nhu cầu kiểm tra định kỳ cao, yêu cầu về một công cụ hỗ trợ thi trắc nghiệm đáng tin cậy ngày càng trở nên cấp thiết.

Song song với xu hướng đó, môn học **Kỹ thuật Lập trình (MI3310)** tại Khoa Toán – Tin, Đại học Bách khoa Hà Nội đặt ra mục tiêu rèn luyện sinh viên không chỉ về kỹ năng viết mã mà còn về tư duy thiết kế phần mềm có cấu trúc. Một trong những yêu cầu quan trọng của môn học là sinh viên phải tự cài đặt các cấu trúc dữ liệu cơ bản bằng tay, không phụ thuộc vào thư viện chuẩn [?]. Điều này tạo ra một bối cảnh học tập lý tưởng để kết hợp nhu cầu thực tiễn với mục tiêu giáo dục: xây dựng một hệ thống thi trắc nghiệm hoàn chỉnh hoàn toàn bằng C++ thuần.

1.2 Lý do lựa chọn bài toán

Nhóm lựa chọn đề tài “*Xây dựng chương trình hỗ trợ thi trắc nghiệm đơn giản*” xuất phát từ ba lý do chính.

Thứ nhất, bài toán có tính thực tiễn rõ ràng. Hệ thống quản lý câu hỏi, sinh đề, đếm giờ và chấm điểm là những nghiệp vụ xuất hiện trong hầu hết mọi hệ thống giáo dục. Việc xây dựng một phiên bản đơn giản giúp nhóm tiếp cận quy trình phân tích – thiết kế – cài đặt theo đúng vòng đời phát triển phần mềm [?].

Thứ hai, bài toán đòi hỏi vận dụng đồng thời nhiều kỹ thuật lập trình nâng cao: quản lý bộ nhớ động với `new/delete`, cấu trúc danh sách liên kết đơn, thuật toán xáo trộn ngẫu nhiên, xử lý File I/O, lập trình hướng đối tượng và tổ chức mã nguồn theo module. Đây chính xác là tập hợp các chủ đề cốt lõi mà môn học hướng đến [?].

Thứ ba, ràng buộc không dùng STL container buộc nhóm phải thực sự hiểu cách các cấu trúc dữ liệu hoạt động ở cấp độ con trỏ, thay vì chỉ sử dụng chúng như những hộp đen. Đây là kinh nghiệm học tập không thể thay thế đối với sinh viên kỹ thuật.

1.3 Mô tả bài toán

Trong thực tế giảng dạy, việc tổ chức một bài kiểm tra trắc nghiệm thủ công đòi hỏi giáo viên phải thực hiện nhiều bước lặp đi lặp lại: soạn đề, in ấn, phát đề, thu bài, chấm điểm và tổng hợp kết quả. Với lớp học đông sinh viên, quy trình này tiêu tốn nhiều thời gian và dễ phát sinh sai sót.

Bài toán đặt ra là xây dựng một chương trình phần mềm chạy trên nền Terminal có khả năng thay thế hoàn toàn quy trình thủ công trên. Hệ thống cần đáp ứng ba nghiệp vụ chính:

1. **Quản lý ngân hàng câu hỏi:** Admin có thể thêm, xóa câu hỏi trắc nghiệm với 4 lựa chọn và đáp án đúng. Dữ liệu được lưu trữ bền vững trong file văn bản, đảm bảo không mất mát khi tắt chương trình.
2. **Tổ chức thi:** Sinh viên đăng nhập, chọn số lượng câu và thời gian, được cấp một đề thi sinh ngẫu nhiên từ ngân hàng câu hỏi. Thứ tự câu hỏi và thứ tự các đáp án được xáo trộn độc lập cho mỗi lần thi, cùng với giới hạn thời gian làm bài được đếm ngược theo thời gian thực.
3. **Chấm điểm và báo cáo kết quả:** Sau khi nộp bài — dù do sinh viên chủ động hoặc do hết giờ — hệ thống tự động chấm điểm, hiển thị kết quả chi tiết từng câu và lưu lịch sử để có thể truy xuất lại sau.

Toàn bộ chương trình được cài đặt bằng C++ (chuẩn C++17), không sử dụng các container của thư viện STL, mà thay vào đó tự cài đặt cấu trúc dữ liệu bằng con trỏ để đáp ứng yêu cầu kỹ thuật của môn học.

1.4 Mục tiêu của chương trình

Chương trình hướng đến các mục tiêu cụ thể sau:

- Xây dựng hệ thống thi trắc nghiệm hoàn chỉnh, có thể vận hành độc lập trên môi trường Terminal mà không phụ thuộc vào kết nối mạng hay giao diện đồ họa.
- Đảm bảo tính công bằng và ngẫu nhiên trong quá trình ra đề thông qua thuật toán xáo trộn Fisher-Yates [?].
- Cung cấp trải nghiệm thi sát với thực tế: có giới hạn thời gian, có cơ chế tự động thu bài và phản hồi kết quả tức thì sau khi nộp bài.
- Lưu trữ và truy xuất lịch sử thi của từng sinh viên qua nhiều lần thi, phục vụ mục đích theo dõi tiến độ học tập.
- Vận dụng và củng cố các kiến thức nền tảng của môn Kỹ thuật Lập trình: con trỏ, cấu trúc dữ liệu tự cài đặt, quản lý bộ nhớ động, xử lý file I/O, lập trình hướng đối tượng và tổ chức chương trình theo module [?].

1.5 Phạm vi thực hiện

Trong khuôn khổ bài tập lớn môn học, chương trình được giới hạn phạm vi như sau:

- **Môi trường thực thi:** Terminal / Command Prompt trên hệ điều hành Windows (MinGW / g++) hoặc Linux/macOS (g++). Không có giao diện đồ họa (GUI).

- **Người dùng:** Hai vai trò — Admin (quản trị viên) và Sinh viên (người thi). Xác thực bằng tên đăng nhập và mật khẩu đơn giản, không mã hoá.
- **Dữ liệu:** Câu hỏi và lịch sử thi được lưu trên file văn bản cục bộ (`questions.txt`, `history.txt`), không sử dụng hệ quản trị cơ sở dữ liệu.
- **Câu hỏi:** Dạng trắc nghiệm một đáp án đúng trong bốn lựa chọn (A, B, C, D). Chưa hỗ trợ câu hỏi tự luận, câu hỏi có hình ảnh hay nhiều đáp án đúng.
- **Đề thi:** Số lượng câu hỏi và thời gian làm bài được sinh viên tự cấu hình khi bắt đầu mỗi lần thi; không phân loại theo môn học hay độ khó.

1.6 Công nghệ và ngôn ngữ lập trình sử dụng

Thành phần	Chi tiết
Ngôn ngữ lập trình	C++ (chuẩn C++17)
Trình biên dịch	MinGW g++ (Windows) / g++ (Linux, macOS)
Thư viện sử dụng	<code><iostream></code> , <code><fstream></code> , <code><sstream></code> , <code><string></code> , <code><cstdlib></code> , <code><ctime></code> , <code><cctype></code> , <code><iomanip></code> , <code><limits></code>
Môi trường phát triển	Visual Studio Code với extension C/C++ (Microsoft)
Quản lý dự án	ClickUp (phân công nhiệm vụ), GitHub (quản lý mã nguồn)
Kiểm thử bộ nhớ	Valgrind (Linux) / DrMemory (Windows)

Bảng 1.1: Công nghệ và công cụ sử dụng trong dự án

Nhóm tuân thủ nguyên tắc không sử dụng biến toàn cục — mọi dữ liệu đều được truyền qua tham số hàm hoặc quản lý trong các class — và không sử dụng các container STL như `std::vector`, `std::list`. Cấu trúc danh sách liên kết đơn được tự cài đặt để lưu trữ câu hỏi và lịch sử thi, nhằm rèn luyện kỹ năng làm việc trực tiếp với bộ nhớ động [?].

1.7 Phân công công việc

1.7.1 Thành viên thực hiện

STT	Họ và tên	MSSV	Lớp
1	Nguyễn Mạnh Hùng	202419063	Hệ thống thông tin quản lý 02
2	Nguyễn Đức Thành	202419099	Hệ thống thông tin quản lý 02
3	Nguyễn Hồng Vân	202419119	Hệ thống thông tin quản lý 02

Bảng 1.2: Danh sách thành viên nhóm

1.7.2 Phân công chi tiết nhiệm vụ

Thành viên	Vai trò	Nhiệm vụ cụ thể
Thành	System Architect	Xây dựng giao diện Menu trên Terminal; điều hướng luồng Admin và Sinh viên; xử lý bất lỗi nhập liệu (validate input); viết file <code>main.cpp</code> , ráp nối các module; kiểm thử toàn hệ thống (cross-testing); đóng gói báo cáo cuối.
Hùng	Algorithm Engineer	Xây dựng luồng cốt lõi bài thi; cài đặt thuật toán Fisher-Yates xáo trộn câu hỏi và đáp án; xử lý module đếm giờ đếm ngược thời gian thực; cơ chế tự động thu bài khi hết giờ; logic tính điểm và tổng hợp kết quả.
Vân	Data Engineer	Cài đặt cấu trúc danh sách liên kết đơn cho ngân hàng câu hỏi và lịch sử thi; quản lý cấp phát và giải phóng bộ nhớ (tránh Memory Leak); xử lý File I/O: đọc/ghi <code>questions.txt</code> và <code>history.txt</code> với ký tự phân tách .

Bảng 1.3: Phân công nhiệm vụ giữa các thành viên

Quá trình phối hợp được thực hiện qua **GitHub** (quản lý mã nguồn theo nhánh) và **ClickUp** (theo dõi tiến độ nhiệm vụ). Quy tắc chung được thống nhất: mỗi thành viên chỉ đẩy code lên khi đã tự biên dịch thành công với **0 lỗi** trên máy cá nhân, tuyệt đối không đẩy code đang lỗi cú pháp ảnh hưởng đến tiến độ chung của nhóm.

Chương 2

Cơ sở lý thuyết

Chương này trình bày các kiến thức nền tảng được vận dụng trong quá trình xây dựng chương trình, bao gồm: nguyên lý thiết kế chương trình theo module, cấu trúc dữ liệu danh sách liên kết đơn, kỹ thuật quản lý bộ nhớ động, thuật toán xáo trộn Fisher-Yates, xử lý File I/O và lập trình hướng đối tượng trong C++.

2.1 Nguyên lý thiết kế chương trình theo module

2.1.1 Tư tưởng module hoá (Modularity)

Một trong những nguyên lý căn bản nhất của kỹ thuật lập trình hiện đại là *module hoá* — tức là phân chia chương trình lớn thành các đơn vị nhỏ hơn, mỗi đơn vị thực thi một nhiệm vụ cụ thể và độc lập [?]. Theo nguyên lý này, mỗi module (hàm, class, hoặc file mã nguồn) cần thoả mãn ba yêu cầu: có nhiệm vụ rõ ràng và duy nhất, giao tiếp với bên ngoài thông qua một giao diện tường minh (tham số và giá trị trả về), và có thể kiểm thử độc lập.

Lợi ích của module hoá thể hiện rõ ở bốn phương diện [?]:

- **Dễ đọc:** Mỗi module đủ ngắn để nắm bắt toàn bộ logic trong một lần đọc.
- **Dễ kiểm thử:** Có thể test từng hàm riêng biệt mà không cần khởi chạy toàn bộ hệ thống.
- **Dễ bảo trì:** Khi cần sửa đổi một chức năng, chỉ thay đổi module liên quan mà không ảnh hưởng đến phần còn lại.
- **Tái sử dụng:** Các module được thiết kế tốt có thể dùng lại trong các dự án khác.

Trong dự án này, nguyên lý module hoá được áp dụng thông qua việc phân chia mã nguồn thành bốn module độc lập: **QuestionBank** (quản lý ngân hàng câu hỏi), **Exam** (logic bài thi), **History** (quản lý lịch sử) và **Menu** (giao diện và điều hướng), mỗi module có file header (.h) và file cài đặt (.cpp) riêng.

2.1.2 Thiết kế từ trên xuống (Top-down Design)

Phương pháp thiết kế từ trên xuống bắt đầu bằng việc phác hoạ toàn bộ chương trình ở mức tổng quan, sau đó tinh chỉnh dần xuống mức chi tiết [?]. Cụ thể, quy trình gồm ba bước lặp:

1. Phác thảo hàm `main()` bằng giả ngữ (pseudocode), mô tả *cái gì* cần làm chứ không phải *làm như thế nào*.
2. Với mỗi tác vụ phức tạp trong giả ngữ, thay thế bằng lời gọi hàm và định nghĩa hàm đó ở bước tiếp theo.
3. Lặp lại quá trình ở mức chi tiết hơn cho đến khi toàn bộ các hàm được định nghĩa đầy đủ.

Phương pháp này giúp kiểm soát độ phức tạp của chương trình ngay từ đầu, tránh tình trạng viết code đến đâu thiết kế đến đó (bottom-up không có kế hoạch) dẫn đến mã nguồn rối và khó bảo trì [?].

2.1.3 Phong cách lập trình tốt

Mã nguồn tốt không chỉ là mã nguồn chạy đúng, mà còn là mã nguồn dễ đọc, dễ hiểu và dễ bảo trì [?]. Các nguyên tắc phong cách lập trình được nhóm tuân thủ trong dự án này bao gồm:

- **Đặt tên gọi nhớ:** Tên biến và hàm phản ánh rõ mục đích (ví dụ: `numberOfQuestions`, `shuffleAnswers`, `generateRandomSet`).
- **Chú thích có ý nghĩa:** Chú thích cho từng đoạn mã (không phải từng dòng), mô tả *tại sao* và *cái gì*, không phải *như thế nào* khi code đã tự giải thích.
- **Nhất quán trong cách lề (indentation):** Toàn bộ mã nguồn dùng 4 dấu cách cho mỗi cấp lề.
- **Không dùng biến toàn cục:** Mọi dữ liệu được truyền qua tham số, tuân thủ nguyên tắc bao đóng (encapsulation) [?].
- **Không vá vúi mã xấu:** Khi phát hiện lỗi thiết kế, viết lại thay vì chắp vá [?].

2.2 Con trỏ và quản lý bộ nhớ động trong C++

2.2.1 Bộ nhớ tĩnh và bộ nhớ động

Trong C++, bộ nhớ được chia thành hai vùng chính: *stack* (ngăn xếp) và *heap* (đống). Biến cục bộ được cấp phát tự động trên stack và giải phóng khi hàm trả về. Ngược lại, bộ nhớ động được cấp phát trên heap thông qua toán tử `new` và phải được lập trình viên tường minh giải phóng bằng `delete` (hoặc `delete[]` với mảng) [?].

```

1 // Cấp phát mảng động
2 Question* questionList = new Question[numberOfQuestions];
3 char*      userAnswers  = new char[numberOfQuestions];
4
5 // Su dụng ...
6
7 // Giải phóng (bắt buộc, tránh Memory Leak)
8 delete[] questionList;
9 delete[] userAnswers;
```

Listing 2.1: Cấp phát và giải phóng mảng động trong C++

2.2.2 Rò rỉ bộ nhớ (Memory Leak)

Rò rỉ bộ nhớ xảy ra khi chương trình cấp phát bộ nhớ động nhưng không giải phóng sau khi sử dụng xong [?]. Trong một chương trình chạy dài, hiện tượng này dẫn đến cạn kiệt bộ nhớ hệ thống. Để phòng tránh, nhóm tuân thủ quy tắc: mỗi `new` phải có một `delete` tương ứng, và việc giải phóng được thực hiện trong Destructor của class.

2.2.3 Constructor và Destructor

Constructor được gọi tự động khi một đối tượng được khởi tạo, còn Destructor được gọi khi đối tượng ra khỏi phạm vi hoặc bị xóa. Đây là cơ chế RAII (Resource Acquisition Is Initialization) của C++, đảm bảo tài nguyên luôn được quản lý đúng đắn [?]. Trong dự án, Destructor của `QuestionBank`, `HistoryManager` và `Exam` đều chịu trách nhiệm giải phóng toàn bộ bộ nhớ động đã cấp phát.

```

1 // Constructor: Cấp phát động mảng Question[N]
2 Exam::Exam(int n, int t) {
3     numberOfQuestions = n;
4     timeLimitSec = t;
5     questionList = new Question[numberOfQuestions];
6     userAnswers = new char[numberOfQuestions];
7     for (int i = 0; i < numberOfQuestions; i++)
8         userAnswers[i] = '-';
9 }
10
11 // Destructor: Giải phóng bộ nhớ động
12 Exam::~Exam() {
13     delete[] questionList;
14     delete[] userAnswers;
15 }

```

Listing 2.2: Constructor và Destructor của lớp Exam

2.3 Danh sách liên kết đơn (Singly Linked List)

2.3.1 Khái niệm và cấu trúc

Danh sách liên kết đơn là một cấu trúc dữ liệu tuyến tính trong đó mỗi phần tử (node) lưu trữ một giá trị dữ liệu và một con trỏ trỏ đến node kế tiếp [?]. Node cuối cùng có con trỏ `next` bằng `nullptr`. Danh sách được quản lý thông qua hai con trỏ: `head` (trỏ đến node đầu tiên) và `tail` (trỏ đến node cuối cùng).

```

1 struct Question {
2     int id;
3     string content;
4     string answers[4];
5     char correct; // 'A', 'B', 'C' hoặc 'D'
6 };
7
8 struct QuestionNode {
9     Question data;
10    QuestionNode* next;
11 };

```

Listing 2.3: Định nghĩa node danh sách liên kết đơn cho câu hỏi

2.3.2 Ưu điểm so với mảng tĩnh

Khác với mảng có kích thước cố định phải khai báo trước, danh sách liên kết đơn cho phép thêm và xoá phần tử linh hoạt mà không cần cấp phát lại toàn bộ vùng nhớ [?]. Điều này rất phù hợp với ngân hàng câu hỏi, nơi số lượng câu hỏi thay đổi theo thời gian: Admin có thể thêm câu hỏi mới (thêm vào cuối danh sách trong $O(1)$ với con trỏ **tail**) hoặc xoá câu hỏi theo ID (duyệt tuyến tính $O(n)$).

2.3.3 Các thao tác cơ bản

Bảng 2.1 tóm tắt các thao tác cơ bản được cài đặt trong dự án cùng độ phức tạp thời gian tương ứng.

Thao tác	Mô tả	Độ phức tạp
<code>addQuestion(q)</code>	Thêm câu hỏi vào cuối danh sách	$O(1)$
<code>removeQuestion(id)</code>	Xoá câu hỏi theo ID	$O(n)$
<code>getQuestionCount()</code>	Đếm số câu hỏi	$O(n)$
<code>getQuestionAt(index)</code>	Lấy câu hỏi tại vị trí index	$O(n)$
<code>printAll()</code>	In toàn bộ danh sách	$O(n)$
<code>generateRandomSet(n)</code>	Bốc ngẫu nhiên n câu	$O(n)$

Bảng 2.1: Các thao tác trên danh sách liên kết đơn QuestionBank

2.3.4 Cài đặt thêm và xoá node

Thao tác thêm node vào cuối (dùng con trỏ **tail**) có thể thực hiện trong $O(1)$ mà không cần duyệt toàn bộ danh sách. Thao tác xoá node theo ID cần xử lý ba trường hợp: danh sách rỗng, node cần xoá là HEAD, và node cần xoá ở giữa hoặc cuối.

```

1 void QuestionBank::addQuestion(Question q) {
2     QuestionNode* newNode = new QuestionNode();
3     newNode->data = q;
4     newNode->next = nullptr;
5     if (tail == nullptr) {
6         head = newNode; // Danh sách rỗng
7         tail = newNode;
8     } else {
9         tail->next = newNode;
10        tail = newNode;
11    }
12 }
```

Listing 2.4: Thêm node vào cuối danh sách liên kết đơn

2.4 Thuật toán xáo trộn Fisher-Yates

2.4.1 Nguyên lý hoạt động

Thuật toán Fisher-Yates (còn gọi là Knuth Shuffle) là một thuật toán xáo trộn mảng *in-place* cho phân phối đều (uniform distribution), nghĩa là mọi hoán vị của mảng đều có xác suất xuất hiện bằng nhau [?]. Thuật toán hoạt động theo nguyên tắc: duyệt mảng từ cuối về đầu, tại mỗi vị trí i , chọn ngẫu nhiên một vị trí j trong đoạn $[0, i]$ và hoán đổi phần tử tại i và j .

```

1 void Exam::shuffleQuestions(Question arr[], int n) {
2     for (int i = n - 1; i > 0; --i) {
3         int j = rand() % (i + 1);    // j ngẫu nhiên trong [0, i]
4         Question temp = arr[i];
5         arr[i] = arr[j];
6         arr[j] = temp;
7     }
8 }

```

Listing 2.5: Thuật toán Fisher-Yates xáo trộn mảng câu hỏi

2.4.2 Độ phức tạp

Thuật toán Fisher-Yates có độ phức tạp thời gian $O(n)$ và không gian $O(1)$, tốt hơn nhiều so với các cách tiếp cận ngây thơ như chọn phần tử ngẫu nhiên và kiểm tra trùng lặp (có độ phức tạp trung bình $O(n^2)$) [?].

2.4.3 Ứng dụng trong dự án

Trong chương trình, Fisher-Yates được áp dụng ở hai cấp độ. Thứ nhất, xáo trộn thứ tự các câu hỏi trong đề thi để mỗi sinh viên nhận được đề có trình tự câu hỏi khác nhau. Thứ hai, với mỗi câu hỏi, xáo trộn thứ tự bốn đáp án (A, B, C, D) và cập nhật lại trường `correct` để phản ánh đáp án đúng trong sắp xếp mới.

```

1 void Exam::shuffleExam() {
2     shuffleQuestions(questionList, numberOfQuestions);
3     for (int i = 0; i < numberOfQuestions; ++i) {
4         Question& q = questionList[i];
5         // Ghi nhớ nội dung đáp án đúng trước khi xáo
6         string correctText = q.answers[q.correct - 'A'];
7         shuffleAnswers(q.answers, 4);
8         // Cập nhật lại vị trí đáp án đúng sau khi xáo
9         for (int k = 0; k < 4; ++k) {
10             if (q.answers[k] == correctText) {
11                 q.correct = static_cast<char>('A' + k);
12                 break;
13             }
14         }
15     }
16 }

```

Listing 2.6: Xáo trộn đáp án và cập nhật đáp án đúng

2.5 Xử lý File I/O trong C++

2.5.1 Các lớp stream cơ bản

C++ cung cấp thư viện `<fstream>` với hai lớp chính cho File I/O: `ifstream` (đọc file) và `ofstream` (ghi file) [?]. Để xử lý chuỗi văn bản phân tách, nhóm kết hợp với `std::istringstream` từ thư viện `<sstream>` giúp phân tách từng trường dữ liệu theo ký tự `|`.

2.5.2 Định dạng dữ liệu

Nhóm lựa chọn định dạng file văn bản phẳng (plain text) với ký tự phân tách `|` cho cả hai file dữ liệu. Định dạng này có thể đọc bằng bất kỳ trình soạn thảo văn bản nào, để

kiểm tra bằng tay khi debug, và không yêu cầu thư viện bên ngoài [?].

Định dạng file `questions.txt`:

`id|NoiDungCauHoi|DapAnA|DapAnB|DapAnC|DapAnD|DapAnDung`

Định dạng file `history.txt`:

`Username|SoCauDung|TongSoCau|Diem|Timestamp`

2.5.3 Đọc file với xử lý lỗi

Một thực hành tốt khi làm việc với File I/O là luôn kiểm tra xem file có mở thành công không trước khi thao tác, và xử lý từng dòng với cơ chế bẫy lỗi để bỏ qua các dòng không hợp lệ thay vì làm crash toàn bộ chương trình [?].

```

1 bool QuestionBank::loadFromFile(string filename) {
2     ifstream fin(filename);
3     if (!fin.is_open()) {
4         cout << "[Loi] Khong the mo file: " << filename << "\n";
5         return false;
6     }
7     string line;
8     while (getline(fin, line)) {
9         if (line.empty() || line[0] == '#') continue;
10        istringstream ss(line);
11        Question q;
12        string token;
13        if (!getline(ss, token, '|')) continue;
14        q.id = stoi(token);
15        // ... phân tích từng trường theo định dạng
16        addQuestion(q);
17    }
18    fin.close();
19    return true;
20 }
```

Listing 2.7: Đọc file câu hỏi với kiểm tra lỗi từng dòng

2.6 Lập trình hướng đối tượng trong C++

2.6.1 Class, đóng gói và trừu tượng hoá

Lập trình hướng đối tượng (OOP) tổ chức mã nguồn xung quanh các *đối tượng* — các thực thể kết hợp dữ liệu (thuộc tính) và hành vi (phương thức) thành một đơn vị thống nhất [?]. Nguyên lý *đóng gói* (encapsulation) yêu cầu che dấu các chi tiết cài đặt bên trong class, chỉ để lộ ra giao diện công khai cần thiết. Điều này giảm sự phụ thuộc giữa các module và giúp thay đổi cài đặt nội bộ mà không ảnh hưởng đến code bên ngoài.

Trong dự án, cả ba class chính (QuestionBank, HistoryManager, Exam) đều có thành viên `private` (con trỏ `head`, `tail`, mảng `questionList`) chỉ được truy cập thông qua các phương thức `public`.

2.6.2 Tách khai báo và cài đặt (Header/Source)

Quy ước phổ biến trong C++ là tách khai báo (declaration) vào file header `.h` và cài đặt (definition) vào file source `.cpp` [?]. File header chứa signature của các hàm và định nghĩa

struct/class, cho phép các module khác `#include` mà không cần biết chi tiết cài đặt. Cơ chế `#pragma once` hoặc include guard ngăn chặn việc include trùng lặp gây lỗi biên dịch.

Cấu trúc này mang lại nhiều lợi ích: giảm thời gian biên dịch lại khi chỉ thay đổi cài đặt, phân công công việc rõ ràng giữa các thành viên nhóm, và tạo ra một “hợp đồng” (contract) tường minh về giao diện của mỗi module [?].

File Header	File Source	Nội dung
QuestionBank.h	QuestionBank.cpp	Struct <code>Question</code> , <code>QuestionNode</code> ; class <code>QuestionBank</code> với các thao tác thêm, xoá, đọc, ghi
History.h	History.cpp	Struct <code>TestRecord</code> , <code>HistoryNode</code> ; class <code>HistoryManager</code> lưu trữ và truy vấn lịch sử thi
Exam.h	Exam.cpp	Class <code>Exam</code> với logic sinh đề, xáo trộn, đếm giờ và chấm điểm
Menu.h	Menu.cpp	Struct <code>LoginSession</code> ; các hàm giao diện Terminal, đăng nhập, menu Admin, menu Sinh viên

Bảng 2.2: Cấu trúc header/source của các module trong dự án

2.6.3 Validate input và lập trình phòng ngừa

Một chương trình tốt không chỉ hoạt động đúng với đầu vào hợp lệ mà còn xử lý ổn thoả với đầu vào không hợp lệ [?]. Kỹ thuật *lập trình phòng ngừa* (defensive programming) yêu cầu kiểm tra mọi giá trị đầu vào từ người dùng trước khi sử dụng. Trong dự án, hàm `getValidInt()` đọc số nguyên trong đoạn `[min, max]` với vòng lặp bẫy lỗi hoàn toàn: khi người dùng nhập ký tự không phải số, `cin.clear()` và `cin.ignore()` được gọi để xoá trạng thái lỗi và bộ đệm, sau đó yêu cầu nhập lại.

2.7 Tổng kết

Chương này đã trình bày sáu nhóm kiến thức lý thuyết làm nền tảng cho dự án: nguyên lý thiết kế theo module và top-down, con trỏ và quản lý bộ nhớ động, danh sách liên kết đơn, thuật toán Fisher-Yates, xử lý File I/O và lập trình hướng đối tượng trong C++. Bảng 2.3 tóm tắt mối liên hệ giữa từng kiến thức lý thuyết và phần cài đặt tương ứng trong dự án.

Kiến thức lý thuyết	Ứng dụng trong dự án
Thiết kế module / top-down	Phân chia thành 4 module độc lập; <code>main.cpp</code> chỉ điều phối luồng chính
Con trỏ / bộ nhớ động	Mảng <code>Question[]</code> và <code>char[]</code> trong class <code>Exam</code> ; cấp phát bằng <code>new[]</code> , giải phóng trong Destructor
Danh sách liên kết đơn	<code>QuestionBank</code> và <code>HistoryManager</code> lưu trữ dữ liệu động không giới hạn kích thước
Fisher-Yates Shuffle	Xáo trộn thứ tự câu hỏi và đáp án trong <code>Exam::shuffleExam()</code>
File I/O (<code>fstream</code>)	Đọc/ghi <code>questions.txt</code> và <code>history.txt</code> với định dạng phân tách
OOP (class, header/source)	3 class với đóng gói đầy đủ; tách khai báo (<code>.h</code>) và cài đặt (<code>.cpp</code>)
Validate input	Hàm <code>getValidInt()</code> , <code>getValidString()</code> , <code>getYesNo()</code> với vòng lặp bắt lỗi

Bảng 2.3: Tổng kết liên hệ lý thuyết và cài đặt trong dự án

Chương 3

Phân tích Yêu cầu hệ thống

3.1 Khảo sát bài toán

3.2 Yêu cầu chức năng

3.2.1 Quản lý ngân hàng câu hỏi

3.2.2 Cấu hình và sinh đề thi

3.2.3 Tiến hành thi và đếm giờ thời gian thực

3.2.4 Chấm điểm tự động và hiển thị kết quả

3.2.5 Xem lịch sử thi

3.3 Yêu cầu phi chức năng

3.4 Mô tả luồng hoạt động của chương trình

Chương 4

Thiết kế hệ thống

4.1 Kiến trúc tổng thể chương trình

4.1.1 Sơ đồ phân chia module

4.1.2 Luồng dữ liệu giữa các module

4.1.3 Sơ đồ luồng điều hướng Menu

4.2 Thiết kế cấu trúc dữ liệu

4.2.1 Cấu trúc file questions.txt

4.2.2 Cấu trúc file history.txt

4.2.3 Quy tắc định dạng và tách trường dữ liệu

4.3 Thiết kế các module chức năng

4.3.1 Module Menu

Giao diện đăng nhập và xác thực người dùng

Menu Admin: quản lý câu hỏi, xem thống kê

Menu Sinh viên: vào thi, xem lịch sử

Cơ chế validate input và bắt lỗi

4.3.2 Module Ngân hàng câu hỏi

Struct Question và các thuộc tính

Danh sách liên kết đơn lưu câu hỏi

Đọc/ghi file questions.txt

Thêm, sửa, xóa câu hỏi

4.3.3 Module Thi

Thuật toán sinh đề: xáo trộn Fisher-Yates

Xáo trộn thứ tự đáp án A/B/C/D

Xây dựng chương trình hỗ trợ thi trắc nghiệm đơn giản

Module đếm giờ đếm ngược thời gian thực

Chương 5

Cài đặt và xây dựng chương trình

5.1 Cấu trúc thư mục chương trình

Chương 6

Kiểm thử và Đánh giá kết quả

6.1 Chiến lược kiểm thử

6.2 Tình huống kiểm thử

- 6.2.1 Test Case-01: Đăng nhập đúng – Admin
- 6.2.2 Test Case-02: Đăng nhập đúng – Sinh viên
- 6.2.3 Test Case-03: Đăng nhập sai mật khẩu
- 6.2.4 Test Case-04: Thêm câu hỏi mới
- 6.2.5 Test Case-05: Sinh đề và xáo trộn câu hỏi/đáp án
- 6.2.6 Test Case-06: Làm bài và chấm điểm chính xác
- 6.2.7 Test Case-07: Tự động thu bài khi hết giờ
- 6.2.8 Test Case-08: Xem lịch sử thi nhiều lần
- 6.2.9 Test Case-09: Nhập sai định dạng (bẫy lỗi)
- 6.2.10 Test Case-10: Kiểm thử rò rỉ bộ nhớ (Valgrind)

6.3 Hình ảnh chụp kết quả thực thi

6.4 Đánh giá kết quả kiểm thử

Chương 7

Kết luận và hướng phát triển

7.1 Kết quả đạt được

7.2 Tổng kết các kỹ thuật đã được vận dụng

7.2.1 Cấu trúc dữ liệu: Danh sách liên kết đơn

7.2.2 Quản lý bộ nhớ động: con trỏ, new/delete, tránh Memory Leak

7.2.3 Xử lý File I/O: đọc/ghi file text với ký tự phân tách '|'

7.2.4 Thuật toán xáo trộn Fisher-Yates

7.2.5 Lập trình hướng module: tách Header – Source – Data

7.2.6 Lập trình hướng đối tượng (OOP): Class, Constructor, Destructor

7.2.7 Lập trình phòng ngừa

7.2.8 Validate input và xử lý ngoại lệ trên Terminal

7.2.9 Quy chuẩn lập trình: Naming Convention, No Global Variables

7.3 Đánh giá ưu điểm và hạn chế của chương trình

7.4 Hướng phát triển và Nâng cấp tương lai

Tài liệu tham khảo

Phụ lục A

Mã nguồn hàm main()

Phụ lục B

Mô tả các hàm xử lý chính

Phụ lục C

Hướng dẫn cài đặt và sử dụng chương trình